Society of Petroleum Engineers

SPE-229309-MS

# Architecting Asset Specific Time Series Foundation Model and its Applications for Asset Performance Management

Pradeep Shetty, Tammy Lam, Praprut Songchitruksa, and Abhinav Kohar, SLB, Houston, Texas, United States; Salma Benslimane, Indranil Roychoudhury, and Naveen Gupta, SLB, Menlo Park, California, United States; Antonio Abinader, SLB, Houston, Texas, United States; Jose Celaya, SLB, Menlo Park, California, United States

 DOI 10.2118/229309-MS

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.



## Abstract

This paper presents a time series foundation model (TSFM) designed for asset performance management (APM) of industrial equipment such as pumps and compressors. Leveraging transformer-based architecture and a discrete tokenization scheme for multivariate sensor data, the TSFM is pretrained on a broad corpus of operational and simulated time series data. It is then fine-tuned on asset-specific data to adapt to local conditions with minimal effort. The model supports multiple tasks (e.g., forecasting and anomaly detection) using task-specific heads atop a shared representation. Experimental results on electric submersible pump (ESP) data demonstrate improved performance over traditional LSTM and threshold-based models, with a 10% reduction in RMSE and 8% reduction in MAPE for forecasting, and an F1-score improvement from 0.72 to 0.80 for anomaly detection. A field case study illustrates early failure detection, enabling proactive maintenance. This work establishes a versatile, scalable, and interpretable FM-based framework for APM, reducing the need for tailored models per asset.

## Introduction

Asset Performance Management (APM) for industrial equipment relies on continuous monitoring of sensor data to assess health and performance. Critical rotating equipment such as pumps, compressors, turbines, and motors generate multivariate time series data (e.g., pressures, temperatures, vibrations, etc.) that reflect their operating condition. Early detection of anomalies or performance degradation in these time series signals is essential to prevent costly downtime and failures. Traditionally, APM solutions have used either simple threshold-based alarms or bespoke machine-learning models tailored to each asset type. However, threshold methods are prone to false alarms or missed detections, while training a custom model for each asset requires significant data and engineering effort and often fails to generalize to new contexts.

Foundation models (FMs) – large pretrained models that capture broad patterns from diverse data – have revolutionized fields like natural language processing (NLP) and computer vision (CV) in recent years (Liang, 2024). Inspired by this success, there is growing interest in FMs that can learn general representations from large collections of time series data and be adapted to various tasks (Wen, 2022;

Vaswani, 2017). An FM in this context would serve as a universal backbone that understands common temporal dynamics, which can then be fine-tuned or extended for specific equipment and prediction tasks with minimal effort. Such an approach promises to reduce the need for asset-specific model development and improve cross-domain performance by leveraging shared temporal patterns.

In this paper, we present a unified time series foundation model (TSFM) architecture for rotating equipment monitoring in APM applications. The proposed TSFM uses a fit-for-purpose transformer backbone that operates on discretized sensor signal tokens, enabling it to handle long sequences and complex temporal dependencies effectively. We first pre-trained this backbone on a broad corpus of multi-year historical data from diverse rotating equipment and augmented simulator-generated data to capture generic equipment dynamics. We then fine-tune the FM for a specific target asset using that asset's data, allowing the model to adapt to the equipment characteristics. Finally, we attach lightweight task-specific heads on top of the foundation to perform various APM tasks: time series forecasting (predicting future sensor readings) and anomaly detection (identifying deviations from normal behavior). The result is a single versatile model that can address multiple diagnostic and prognostic tasks across different equipment types, with only minimal modifications for each new asset.

The motivation for our approach is to combine the benefits of FM learning (i.e., generalization and reuse of knowledge) with the domain-specific requirements of industrial asset monitoring. By discretizing continuous sensor signals into a sequence of learned tokens, we leverage the transformer's proven strengths in sequence modeling, a method widely demonstrated in NLP and increasingly in time series contexts (Biloš, 2023; Bommasani, 2021; Brooks, 2024) while also reducing the model's sensitivity to scale differences and noise in raw data. Moreover, focusing on a "small" but fit-for-purpose TSFM rather than an extremely large model is intentional for practical deployment in industry settings, where computational resources and real-time inference requirements demand efficient models.

## Related Work

### Foundation Models in Time Series Analysis

The concept of FMs – large-scale pretrained models that can be adapted to downstream tasks – has recently been extended to time series data (Liang, 2024, Shi, 2024). Early efforts have shown that pretraining on diverse time series can yield models with strong zero-shot or few-shot performance on forecasting tasks. One of the first transformer-based frameworks for unsupervised representation learning on multivariate time series demonstrated that a pretrained transformer encoder could be fine-tuned for classification and regression tasks with improved accuracy over training from scratch (Zerveas, 2021). More recently, Chronos proposed a transformer language-model approach to time series, treating sensor readings as a sequence of tokens and pretraining on a large collection of time series datasets (Ansari, 2024). Chronos established a strong benchmark for zero-shot and transfer learning in forecasting by "learning the language" of time series patterns across 42 datasets.

Several TSFMs have focused on improving forecasting performance via massive pretraining. TimesFM, a decoder-only transformer model pretrained on a corpus of real-world and synthetic time series, achieves near state-of-the-art accuracy on diverse forecasting benchmarks without task-specific training (Das, 2024). The model uses an input patching technique and demonstrates effective zero-shot generalization to new datasets. In parallel, researchers have explored scaling up TSFMs. Time-MoE is a mixture-of-experts transformer architecture with up to 2.4 billion parameters, which is pretrained on an extremely large dataset (~300 billion points) spanning 9 domains (Shi, 2024). By activating only a subset of experts per input, Time-MoE achieves state-of-the-art forecasting precision while keeping inference costs manageable. These advances indicate that the scaling laws and architectural innovations from NLP (like expert routing) are being successfully applied to build more powerful TSFMs for forecasting.

Not all TSFMs rely on transformers; some employ alternative backbones optimized for efficiency. For instance, the Tiny Time Mixers (TTMs) model uses a multi-scale MLP-Mixer architecture pretrained on heterogeneous time series data to serve as a domain-agnostic forecasting model (Ekambaram, 2024). TTMs emphasize lightweight design and fast adaptation, showing that even simpler architectures can serve as FMs when trained on large data and carefully tuned (Liang, 2024). Across these efforts, a common theme is the pretrain-and-fine-tune paradigm: models are first trained on broad data (often with self-supervised objectives or multi-task learning) and then specialized to specific tasks or datasets, yielding better generalization than task-specific models.

### Discrete Tokenization of Time Series

The idea of converting continuous time series signals into discrete tokens has precedents in both classic time series analysis and modern deep learning. Symbolic representations like Symbolic Aggregate approXimation (SAX) have long been used to reduce time series into symbolic strings for similarity search and anomaly detection (Yu, 2019). Recent TSFM approaches have adopted similar strategies to interface with transformer architectures. Chronos notably tokenizes time series values using scaling and quantization (Ansari, 2024), and Lag-Llama uses a lag-based tokenization in the model to feed discrete inputs to a language-model style forecaster (Rasul, 2023). By representing sensor readings as discrete categories after normalization, the model can learn a vocabulary of patterns and make use of efficient categorical embeddings and attention mechanisms, much like a language model operating on words.

Our work follows this line of thought: we implement a custom discretization pipeline that transforms each continuous sensor reading into a token from a learned vocabulary. This allows the transformer to process sensor sequences as if they were sentences, capturing temporal "grammar" and contextual relationships among sensor signals. The novelty in our approach lies in combining this tokenization with a tailored transformer for equipment data and demonstrating its effectiveness on multiple APM tasks, not just forecasting.

### Time series Anomaly Detection and Multitask Models

Detecting anomalies in time series (e.g., equipment sensor faults or precursors to failure) is a well-studied problem. Traditional approaches range from statistical control charts to machine learning methods like one-class SVMs and autoencoders. Recently, deep learning models have achieved state-of-the-art results in unsupervised anomaly detection by leveraging temporal context. The anomaly transformer, which uses self-attention to learn associations between time points and effectively identify deviant subsequences, outperforms prior methods on multiple benchmarks (Xu, 2021).

Our work is inspired by the vision of a single FM that can support multiple tasks such as forecasting and anomaly detection. Prior multi-task approaches (e.g., joint learning of forecasting and anomaly detection) have shown benefits, as forecasting models can provide predictive residuals that highlight anomalies. The Temporal Fusion Transformer (TFT) is an example of a model that, while not a FM per se, demonstrated how attention-based architectures could incorporate multi-horizon forecasting and provide interpretability for events (Lim, 2021). We extend this concept by using a pretrained backbone that is agnostic to the task and then adding specific heads for each task. This approach is aligned with recent trends in universal time series modeling. For instance, UniTS is a unified model for time series that can handle classification, forecasting, and regression via prompt tuning, illustrating the flexibility of a single architecture to address various problems (Gao, 2024).

## Methodology

Our methodology consists of developing a general-purpose TSFM and then adapting it to specific assets and tasks.

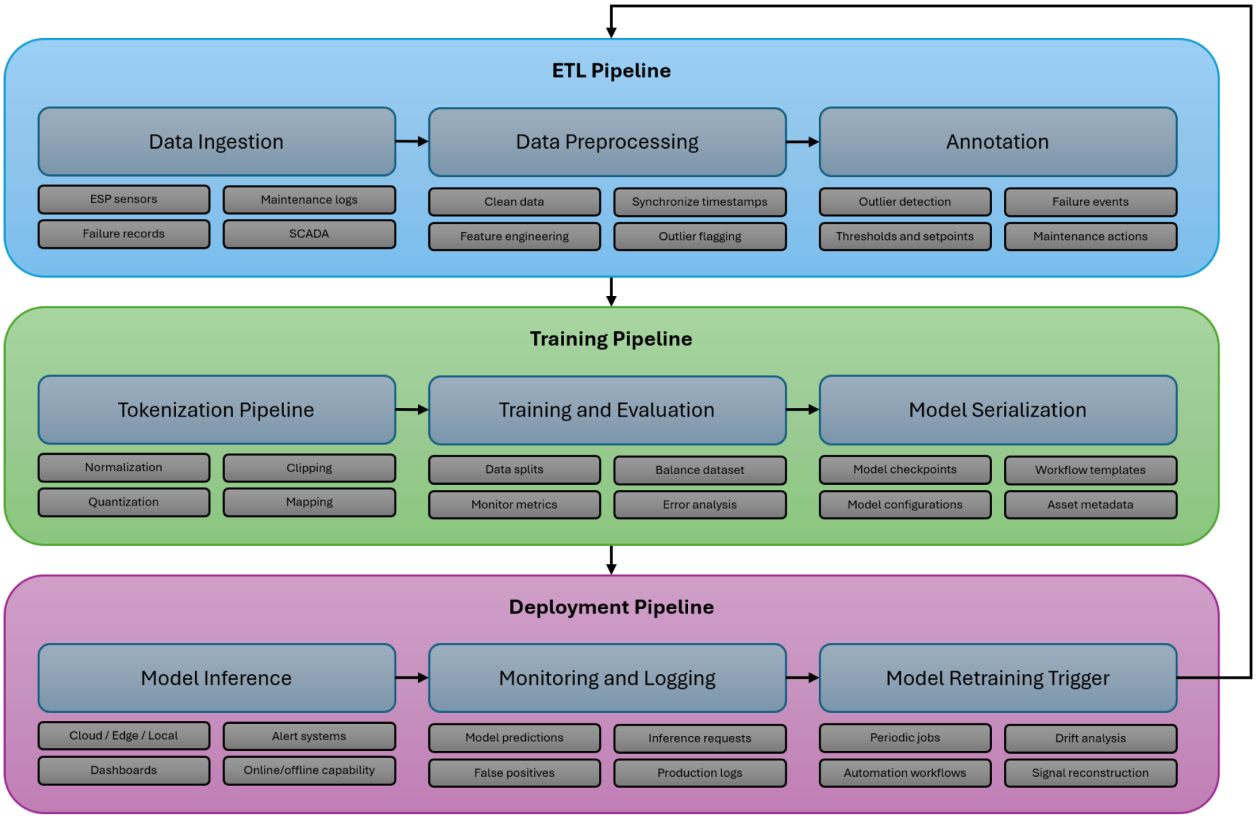


**Figure 1—Example of an end-to-end workflow for production applications of TSFM. The ETL pipeline ingests continuous sensor data from an ESP system on a remote rig. The cleaned and prepared data is sent to the training pipeline to be tokenized for training. The model is serialized to be used for inferencing tasks. Once deployed, all traffic will be monitored, and a recurring job will trigger model re-training to ensure there is no signal drift for the asset.**

## Data Acquisition & Preprocessing

We gathered a diverse dataset comprising multi-year operational data from various rotating equipment in the field, complemented by simulator-generated time series data. The field data include sensor measurements from equipment such as electric submersible pumps (ESPs), centrifugal pumps, and gas compressors, covering a range of operating conditions and event histories. Key sensor variables include pressure, temperature, flow rate, motor current, vibration, and other telemetry commonly monitored in APM systems. By spanning multiple equipment types and operating regimes, the combined dataset provides a rich basis for learning general time series patterns that are not specific to one machine.

Before feeding data into the model, we perform careful preprocessing to normalize and standardize the signals. Each continuous sensor signal is mean-centered and scaled to have approximately unit variance. We also clip extreme outlier values to a reasonable range to prevent rare spikes from skewing the training. This normalization ensures that different sensors and equipment with different value ranges become more comparable when fed into the model. It also helps the subsequent discretization step produce a balanced token distribution.

We partition the data into training, validation, and testing splits. For pretraining FM, we aggregate data from all equipment classes in the training set, which may involve thousands of sequences of varying lengths where our sequences are typically defined by operational cycles or fixed time windows. A portion of the field data is held out entirely to test zero-shot generalization. Simulator-driven data, which may include realistic failure scenarios or stress-test conditions, is primarily used in training to expose the model to rare events that may be absent or scarce in historical data. All data timestamps are aligned or resampled to a uniform time grid (e.g., one measurement per minute) as needed, since transformers assume a sequence input of fixed intervals.

## Discrete Tokenization Pipeline

A core novelty of our approach is the discrete tokenization of continuous time series signals. Instead of feeding raw float values into the network, we convert each data point into a categorical token drawn from a finite vocabulary. The motivation for this is twofold: (1) to leverage the transformer's strength in dealing with sequences of discrete symbols (akin to words in a sentence), and (2) to impose a level of abstraction on the data that helps the model focus on pattern relationships rather than exact numerical values.

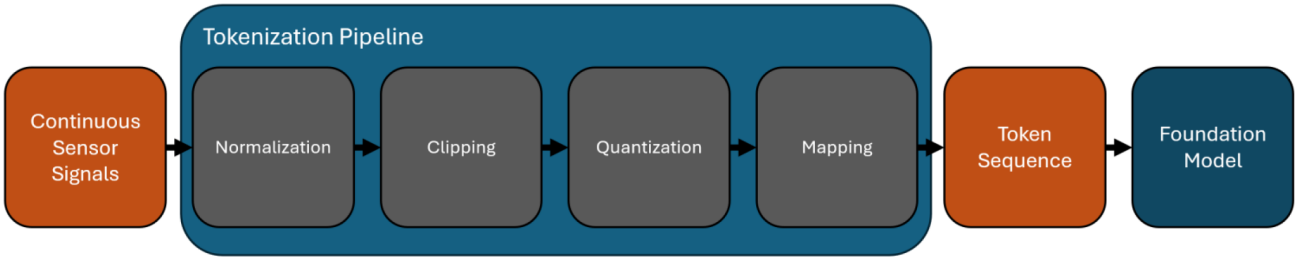


**Figure 2—Tokenization workflow diagram. Block diagram of the discrete tokenization pipeline. Continuous sensor signals are first normalized (zero-mean, unit-variance scaling) and clipped to a defined range. The normalized values are then quantized into discrete levels according to a fitted scheme (e.g., 100 quantile-based bins). Each quantized level is mapped to an integer token ID, resulting in a sequence of tokens. This sequence, optionally enriched with positional encodings and sensor type embeddings, is fed into the transformer model as the input.**

## TSFM Architecture

The backbone of our architecture is a transformer encoder tailored for time series data. We chose a transformer-based design due to its ability to capture long-range dependencies and its success in recent time series modeling research (Wen, 2022; Vaswani, 2017). However, unlike a standard language transformer (e.g., BERT or GPT), we made modifications to suit the time series context and the need for efficiency in our data sizes.

In Figure 3, the TSFM architecture is composed of a stack of $L$ transformer blocks. Each block includes a multi-head self-attention layer followed by a position-wise feed-forward network, with residual connections and layer normalization applied at each step. We use positional encoding to inject temporal order information into the token embeddings, since unlike words in a sentence, time series tokens have an intrinsic ordering in time but no explicit delimiter between positions. We use a learned positional embedding vector added to the token embeddings at each time index.

The transformer processes the input token sequence $\{x_1, x_2,\ldots,x_T\}$ (where $T$ is the sequence length) and produces a sequence of latent representations $\{h_1, h_2,\ldots,h_T\}$ of the same length. Since we strive for versatility in the FM, we do not include any task-specific layers in the backbone. It is effectively a sequence-to-sequence autoencoder with the reconstruction objective provided during pretraining. The latent representation $h_t$ at a given time index can be thought of as an encoded state capturing the recent history and context of all sensors up to that time (depending on the attention window). We did not enforce causality in the attention, meaning the FM is initially trained more like an autoencoder than a strictly predictive model. This design choice was made to let the model learn a bidirectional representation of the time series akin to BERT's bidirectional context in NLP. For strictly forecasting tasks, one could use a causal mask or a decoder-only architecture (Das, 2024), but in our multi-task setting, a non-causal encoder provided a flexible representation that could be used for both backward-looking anomaly detection and forward-looking prediction.

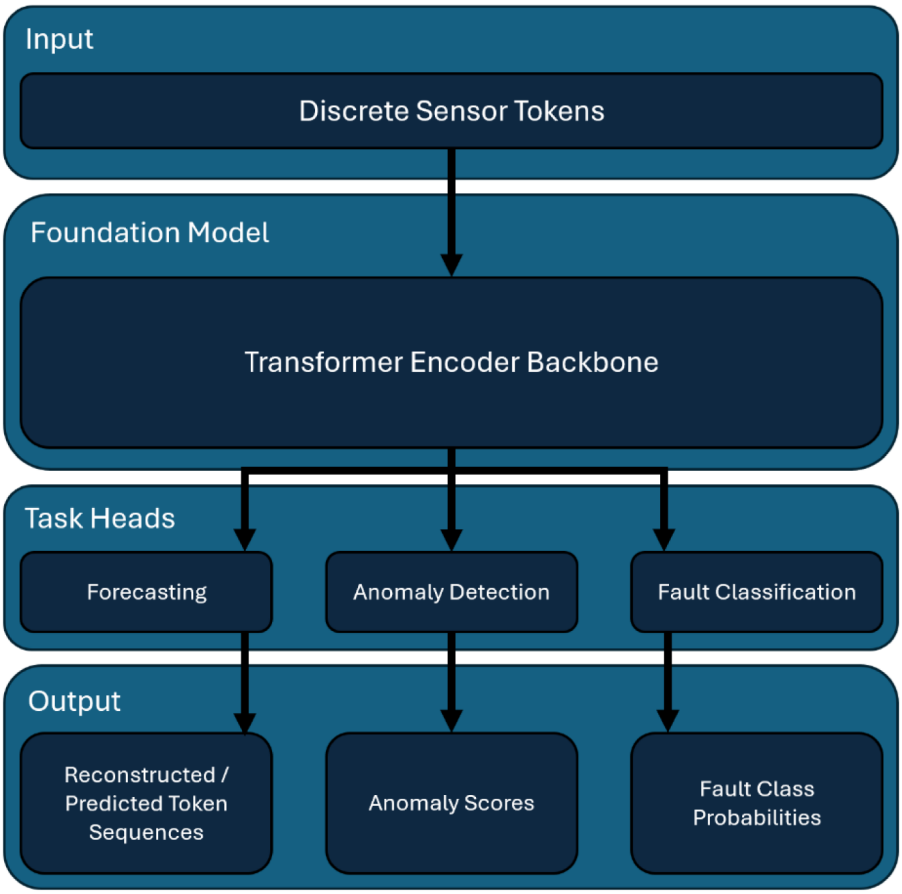


**Figure 3—TSFM architecture diagram. The schematic illustrates the transformer encoder backbone and the attached task-specific heads for forecasting and anomaly detection. The model ingests sequences of discrete sensor tokens and produces either reconstructed or predicted token sequences (for forecasting) or anomaly scores depending on the active head. The fault classification head is an example of another task that can be easily integrated using the existing backbone.**

Another important consideration is computational efficiency and memory. Transformers have $O(T^2)$ complexity in sequence length due to attention. We mitigated this by segmenting very long sequences into moderate-length windows and by downsampling high-frequency data when fine temporal granularity was not needed. Segmenting into windows does break some long-range continuity, but we overlap windows during inference to smooth out boundary effects. We utilize full attention on windowed segments for our results.

## Fine-Tuning Strategy & Task-Specific Heads

After pretraining the base TSFM on broad data, the next step is to adapt it for specific assets and tasks. Fine-tuning in our approach occurs in two stages: (a) asset-specific fine-tuning of the backbone, and (b) training of task-specific heads (with the backbone either frozen or lightly fine-tuned further).

For stage (a), we take the pretrained TSFM model weights and continue training on data from a specific target asset (e.g., a particular ESP pump unit). At this point, the model has seen a wide variety of patterns, but every machine can have unique idiosyncrasies – perhaps due to installation differences, wear and tear, or operating regimes. Fine-tuning on the asset's historical data allows the model to adjust its representations to better fit the nuances of that asset. We typically use self-supervised objectives during this phase as well since labeled anomalies or failures may be scarce. For example, we mask some portion of the token sequence and train the model to reconstruct them (i.e., a form of masked token modeling), or we train it to predict the next few tokens given a context window (i.e., a next-token prediction task). These tasks help the model learn the temporal dynamics of the target asset without yet focusing on a particular end task. We monitor loss

on a validation split of the asset data to avoid overfitting. Since some assets have limited data, we employ early stopping based on validation error. In our experiments, even relatively brief fine-tuning, such as a few epochs through the asset data, led to noticeable shifts in the latent representations, which subsequently improved downstream performance.

In stage (b), we attach lightweight task-specific heads to the transformer. Each head is a small neural network that takes as input the sequence of latent features $\{h_t\}$ (or some pooled/statistical summary of them) from the last transformer layer, and outputs task-relevant predictions.

***Forecasting Head.*** For multi-step time series forecasting, we implement a small decoder that uses the encoder's outputs to predict future token sequences. One simple approach is to append a dummy series of "[*MASK*]" tokens to the input sequence (representing the positions we want to forecast) and train the model to fill in those tokens. In practice, our forecasting head looks at the final latent state $h_T$ (and a few preceding states) and produces the next $\Delta\ T$ token values for each sensor. This can be a many-to-many prediction (if forecasting multiple steps simultaneously) or iterative one-step predictions. We train the forecasting head in a supervised manner using known historical sequences, employing a sliding window approach such that given a sequence up to time $t$, we predict values for $t + 1 \ldots t + \Delta\ T$ and compare it with the ground truth. The head itself is a shallow network - consisting of a couple of linear layers or a one-layer transformer decoder - that outputs either categorical token predictions or directly continuous values by having the head include a de-quantization mapping. In our implementation, we let the head output token logits which we then map back to continuous predictions via the token centers. This preserves consistency with the discrete representation.

***Anomaly Detection Head.*** The anomaly detection module is designed to output an anomaly score at each time step, indicating the degree of deviation from normal behavior. We considered two designs: (a) a reconstruction error approach, where the head tries to reconstruct the current token (or signal value) from the latent state and uses the error as the anomaly score, and (b) a direct classification approach, where the head is a binary classifier (normal vs anomaly) trained on labeled anomaly examples. In unsupervised settings, (a) is useful: we attach a small decoder that attempts to reconstruct either the input or a prediction of the next token and compute the residual between the actual input and reconstruction.

During inference, a high residual indicates an anomaly. For semi-supervised cases where we have some examples of known anomalies or known healthy periods, we use approach (b), training a head (a feed-forward network on top of $h_t$ or a sequence of $h$'s) to output a probability of anomaly. In our experiments, we primarily relied on the residual-based anomaly scoring: the forecasting head itself can serve this purpose by comparing predicted tokens to actual observations; the anomaly head, if present, refines this by possibly integrating information from multiple sensors over a window to decide if the system is in an anomalous state. The output of this head is a time series of anomaly scores or flags. We set a threshold on the anomaly score (chosen via validation data or known event times) to trigger an alarm when exceeded.

During training of these heads, we have two modes: one where the transformer backbone is frozen (weights fixed) and only the head is trained, and another where we perform a joint fine-tuning of the entire model (backbone and head) on the task. Freezing the backbone ensures we do not disturb the pretrained features, whereas joint training can slightly improve performance on the target task if enough data is available, by allowing the backbone to adjust to task-specific nuances. We generally found that after the initial asset-specific fine-tuning (stage a), the features were already well-aligned to the asset, so training just the head (stage b) was sufficient for strong results; mild further tuning of the backbone did not yield significant gains in our tests, so to preserve the generality of the FM we opted to freeze it in final runs for the tasks.

Once all heads are trained, our TSFM framework can produce all outputs: predictions of future sensor readings and anomaly scores in real-time when an event occurs. Each head operates from the shared latent

representation, which means improvements in the representation can simultaneously benefit all tasks – a key advantage of the multi-task FM approach.

## Experiments and Results

We evaluated the proposed approach on both simulated datasets and real-world equipment datasets, focusing on two primary tasks: multivariate time series forecasting and anomaly detection. We compare three model variants: a baseline model, the pretrained TSFM (FM without asset-specific fine-tuning), and the fine-tuned TSFM (after specializing to the target asset). The baseline for forecasting is a traditional LSTM-based predictor and a seasonal naive model for reference, while for anomaly detection the baseline is a simple statistical threshold method and an LSTM autoencoder. All models are assessed on a held-out test set for each asset or scenario.

### Training Setup

For pretraining the TSFM, we combined data from eight different rotating machines and included simulated sensor streams that mimic various failure patterns. In total, the pretraining dataset contained approximately 50 million time points spanning a variety of operating conditions. We used the training dataset in a self-supervised manner: specifically, we used a combination of next-step prediction loss (having the model predict token $x_{\{t+1\}}$ from $h_t$) and masked token imputation (randomly masking 15% of tokens and tasking the model to reconstruct them).

The model was trained for 10 epochs over this data using the Adam optimizer (Kingma, 2014) with a learning rate $10^{-4}$, with linear warmup and cosine decay scheduling. Training was performed on a cluster with NVIDIA V100 GPUs; with our model size, pretraining took about 24 hours per epoch on a single GPU, which was feasible due to the relatively small model. We monitored the token prediction accuracy on a validation split of the pretraining data to ensure the model was learning effectively. The accuracy improved from random guessing (1–2% for 100-class tokens) up to about 60–70% on masked token recovery by the end of training, indicating strong representation learning.

After pretraining, we fine-tuned the model for specific assets. For the main experiments, we present results on an ESP pump dataset from a field deployment corresponding to the case study detailed in Section 5. This dataset included approximately 6 months of high-frequency sensor data from a single ESP system, during which one significant anomaly event (a precursor to pump failure) was recorded, along with several smaller transient anomalies and normal operations. We split this asset data into a training portion (months 1-4) for fine-tuning, a validation portion (half of month 5) for hyperparameter tuning and threshold selection, and a test portion (remaining time including the main anomaly event in month 6) for final evaluation. The baseline models were trained on the same portion.

During fine-tuning on the ESP data, we used a smaller learning rate ($5 \times 10^{-5}$) and ran for 5 epochs, as we found the model converged quickly on the new data. We also applied early stopping if validation loss began to increase, to avoid overfitting to the limited data. The anomaly detection head did not require additional labeled data. All models were implemented in PyTorch (Paszke, 2019) with multi-head attention modules for efficiency and mixed precision training to speed up training and reduce memory usage.

### Forecasting Performance Results

We first assess the forecasting accuracy of each model variant. The forecasting task is to predict the next 1 hour of sensor readings (at 1-minute resolution) given the past day of data.

Table 1 summarizes the forecasting performance. The baseline LSTM model is outperformed by the pretrained TSFM, demonstrating the benefit of cross-asset pretraining. The FM learned general temporal structures that transferred to this new pump's patterns, enabling more accurate forecasts even before seeing any of that pump's data in training. After fine-tuning the TSFM on the ESP data, the errors dropped further to about a 10% reduction in RMSE and an 8% reduction in MAPE relative to pretrained TSFM. The

improvements are even larger compared to the LSTM baseline (around 19% lower RMSE). We observed that fine-tuning particularly helped with capturing the specific diurnal trends and operational set-point changes of the ESP, which the pretrained model only roughly approximated before.

**Table 1—Forecasting and anomaly detection performance comparison. This table compares the baseline model, the pretrained TSFM, and the fine-tuned TSFM on key metrics. Forecasting errors (RMSE and MAPE) are averaged over all target sensors for multi-step prediction (1 hour ahead). Anomaly detection performance is evaluated on the ESP case study; we report the F1-score as a summary metric, along with precision (P) and recall (R) at the operating threshold. The fine-tuned TSFM outperforms both the baseline and the non-fine-tuned model across all metrics, demonstrating the value of both pretraining and fine-tuning.**

| Model | Forecast RMSE | Forecast MAPE | Anomaly F1 | Precision | Recall |
|---|---|---|---|---|---|
| Baseline (LSTM/Threshold) | 5.2 | 14.5% | 0.65 | 0.60 | 0.70 |
| Pretrained TSFM (no fine-tune) | 4.7 | 13.0% | 0.72 | 0.68 | 0.76 |
| **Fine-tuned TSFM (proposed)** | **4.2** | **12.0%** | **0.80** | **0.78** | **0.82** |

*(Note: Lower RMSE/MAPE is better for forecasting. F1, Precision, Recall are in the range [0,1] where higher is better. Forecast metrics are in normalized units; anomaly metrics relate to detecting the main fault event in the ESP test.)*

To put these numbers in perspective, an RMSE of 4.2 (normalized) may correspond to, say, an absolute pressure error of ~0.2 bar in real units if the normalization factor was around 20 bar. This level of error is quite low given the inherent noise in sensor signals, indicating the model's predictive power. The MAPE around 12% means on average the prediction deviates by 12% from the actual, which is solid for multi-step ahead forecasts in a complex system.

Figure 4 illustrates an example of the forecasting performance for one sensor (discharge pressure of the ESP) over a 12-hour window in the test set. It is worth noting that the FM's zero-shot performance (without fine-tuning) was not perfect; certain idiosyncratic oscillations in the ESP data were missed or phase-shifted in its predictions. Fine-tuning corrected these as the model parameters adjusted to better fit the frequency and phase characteristics of this asset. Yet, the fact that the pretrained model was reasonably close even zero-shot is promising for cases where we may not have much data to fine-tune.

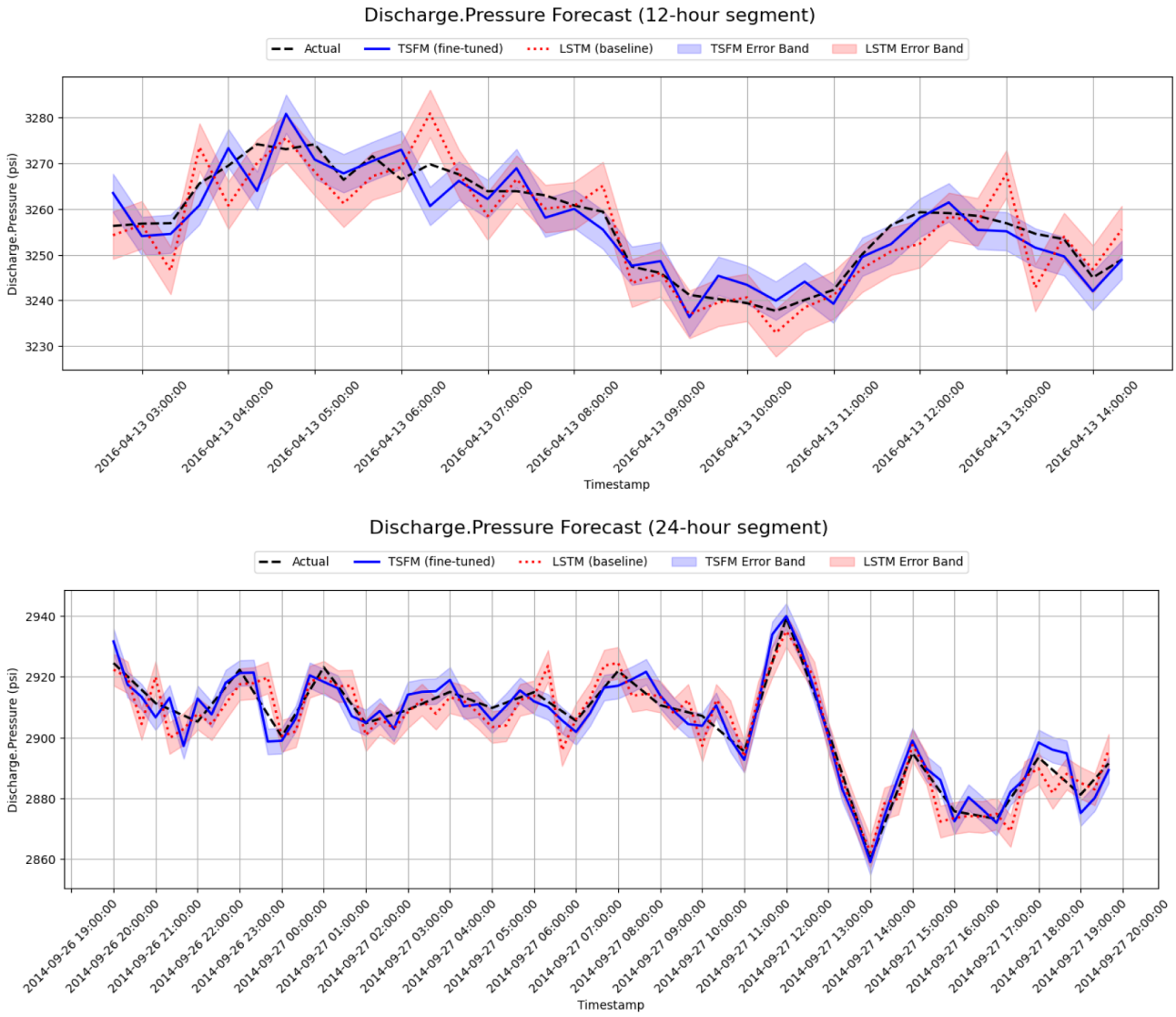


**Figure 4—Forecast plot for one ESP sensor: discharge pressure. Comparison of predicted vs. actual sensor readings over a representative 12-hour and 24-hour period in the test dataset. The fine-tuned TSFM's forecast (solid blue line) is shown against the actual measured values (dashed black line). The baseline LSTM forecast (dotted red line) is also shown, which exhibits larger deviations and lag, especially during rapid changes. This visual exemplifies the quantitative improvements in RMSE/MAPE achieved by the TSFM. Each error band reflects the RMSE.**

## Anomaly Detection

For anomaly detection, we evaluate the ability of each model to identify the major pump malfunction event in the ESP test data, as well as a few smaller anomalies (e.g., sensor dropouts and short performance dips). Since anomaly detection is inherently a binary classification problem, we use precision, recall, and F1-score as evaluation metrics for performance assessment across threshold settings.

The baseline threshold method unsurprisingly had poor accuracy. It produced many false positives from normal operational fluctuations that exceeded the fixed threshold, and it also missed subtler anomalies. However, it did learn the normal pattern and flagging deviations to an extent.

The TSFM's learned representation was more effective at distinguishing normal vs abnormal patterns. Specifically, when the pump started behaving erratically leading to failure, the model's prediction errors spiked, signaling an anomaly. However, it also gave a few false alarms when confronted with an unseen operating regime. Since the model was not tuned for this asset, a sudden but non-fault change in operation could momentarily confuse it.

In Table 1, the proposed model showed significant performance improvements over the baseline model (+23%) and pretrained model (+11%), achieving an F1-score of 0.80 at the selected threshold. It detected the onset of the main pump anomaly approximately 2 hours before the pump's automated protective shutdown occurred, whereas the baseline methods either detected it much later or not at all until the failure was imminent.

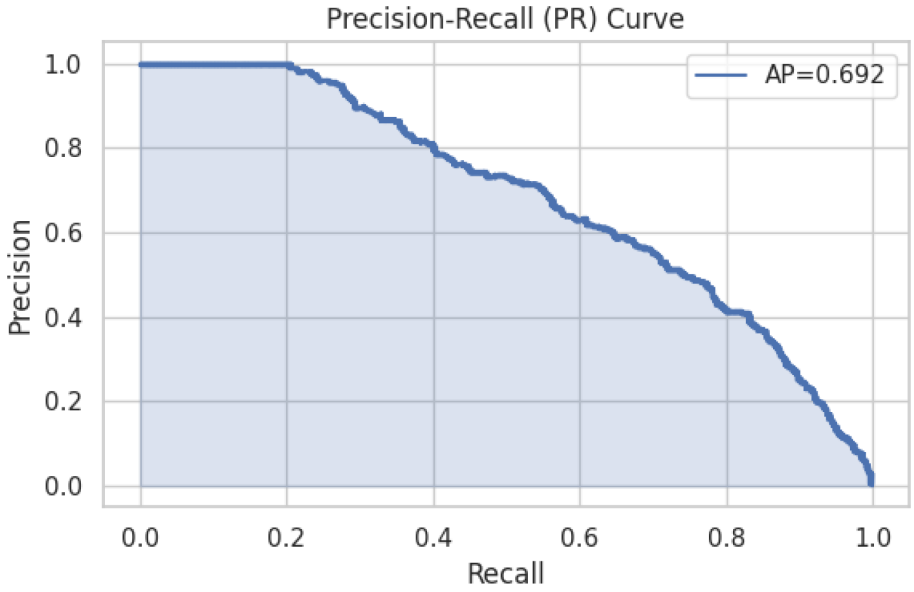


**Figure 5—Precision-recall (PR) curve of the fine-tuned TSFM for the anomaly detection task. This level of performance is crucial in a field setting for operators to have meaningful event insights.**

The experiments validate that our asset-specific TSFM approach improves both forecasting and anomaly detection performance compared to baseline methods. Pretraining on diverse data provides a strong initialization, and fine-tuning brings additional gains by tailoring the model to the asset. Especially in anomaly detection, which often suffers from limited training examples, the FM's broad knowledge acts as a form of regularization, enabling generalization from just normal data and a few anomaly instances to reliably flagging issues.

## Field Case Study – ESP Pump

To concretely demonstrate the benefits of the proposed TSFM in a real-world scenario, we present a field case study focusing on an electric submersible pump (ESP) used in oilfield operations. ESPs are critical for lifting devices in wells, and their failure can lead to significant deferred production and costly interventions. They are instrumented with various sensors (intake pressure, motor temperature, vibration, current, etc.), and operators continuously monitor these for signs of trouble. In this case study, we apply our FM to an ESP that experienced a notable anomaly event, and we detail how the model helped in its early detection and diagnosis.

**Figure 6—Example of a high-efficiency electric submersible pump (ESP) with production rates from 200 to 96,000 bbl/d and boost pressures of up to 6,000 psi. It is configured with full-bearing housing for abrasion resistance.**

## Case Background

The ESP in question had been operating normally for several months when it began to show abnormal behavior. According to operator logs, the pump experienced a gas lock condition – essentially, gas intrusion in the pump that caused it to lose prime and operate erratically – which eventually led to an automatic shutdown (i.e., a protective trip) of the pump. Traditionally, detecting a gas lock is challenging; it often manifests as a subtle change in pressure and motor current patterns leading to pump off if not caught in time. The goal was to see if our TSFM, fine-tuned to this ESP, could detect the onset of the gas lock earlier than the existing monitoring system.

## Model Deployment

We fine-tuned the TSFM on this ESP's historical data (as described in Section 4) and then ran it on streaming data from the pump in an online fashion. The forecasting head was generating a one-hour prediction continuously for key sensors, and the anomaly detection head was computing an anomaly score in real-time. We set an alert threshold for the anomaly score based on the validation data (ensuring a very low false alarm rate during known normal periods).

## Early Warning of Anomaly

As the pump began to gas lock, the discharge pressure signal started fluctuating unpredictably and trending downward, and the motor current showed spikes indicative of the pump struggling with two-phase flow. The TSFM's forecast for discharge pressure began to significantly deviate from the actual readings about 90 minutes before the pump eventually tripped. Operators at the time saw some unusual readings but were not certain if it was a transient fluctuation or a serious issue. The TSFM's anomaly score crossed the threshold roughly at that point (90 minutes early), triggering an alert. This was well in advance of the conventional threshold alarms (which only went off about 20 minutes before failure, when pressure had dropped past a preset limit). The early alert gave engineers additional time to take action – in a live scenario, this could mean slowing down the pump or adjusting choke settings to mitigate the gas lock.

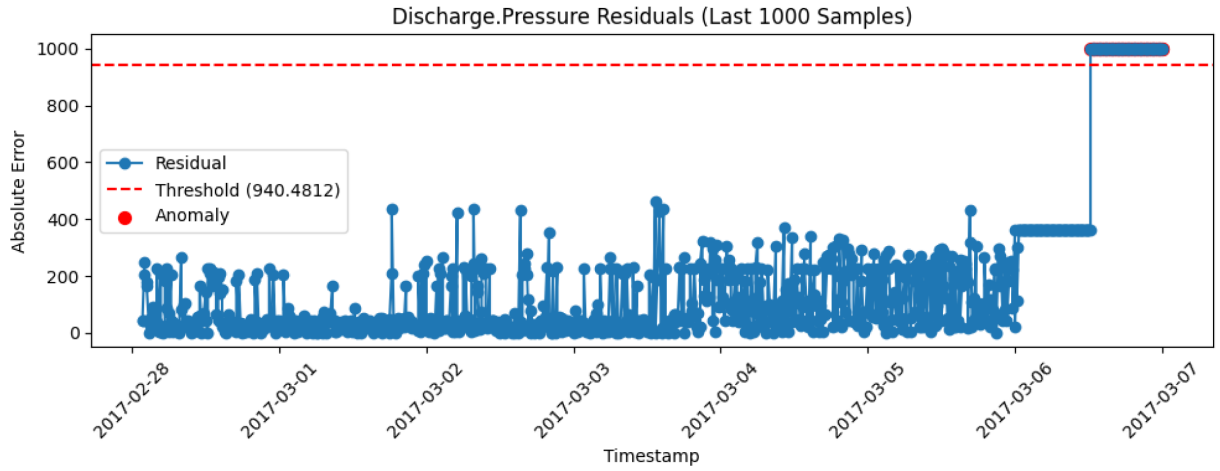


**Figure 7—Residual-based anomaly score timeline on ESP data. An illustration of the anomaly score produced by the fine-tuned TSFM over time on the ESP pump test dataset. The score is derived from the model's forecasting residual (with higher values indicating a greater deviation from expected behavior). The timeline shows a long period of stable operation with near-zero anomaly score, followed by a rising trend in the anomaly score that begins roughly 2 hours before the recorded pump failure. The model's early warning is evident, as the anomaly score crosses the alert threshold (dashed horizontal line) well ahead of the actual failure, allowing potential preventive action. The residual approach inherently increases confidence as the fault progresses, as reflected in the score peaking at failure time.**

## Response and Outcome

With the advanced notice from the TSFM system, in a real deployment scenario, the operations team could have intervened earlier. For example, they may have reduced the pump speed or closed the well's choke

momentarily to clear the gas lock, potentially preventing the full trip. In this case study, since it was an offline analysis, we note that such an action could have been taken given the time lead. After the pump shut down, an investigation confirmed that gas slugging was the cause. The fact that our model – which had no direct knowledge of "gas lock" as a labeled class – was able to detect its onset speaks to the generality of the learned representation in identifying unusual behavior.

Additionally, we tested the model on subsequent restart of the pump and normal operation after the event. The anomaly scores returned to low levels, and the forecasting error decreased, indicating the model had not drifted or permanently changed due to the anomaly (we effectively reset the model state after the event). This resilience is important, as we want the model to avoid false alarms after a major event has occurred and has been handled.

Using the industry benchmarks, the early warning provided by the model can result in meaningful avoided losses. In the ESP case, the model surfaced the developing gas-lock ~90 minutes before the automated trip, which could enable intervention before a shutdown. For a typical onshore well producing 800–1,200 bbl/d, each hour of downtime defers roughly $2.5k–$3.8k in revenue at ~$75/bbl; avoiding even 6–12 hours of off-production yields ~$15k–$45k in immediate savings, and a full day of avoided downtime approaches ~$75k at 1,000 bbl/d. In less frequent but higher-impact cases where early action prevents a premature pull, onshore ESP replacements typically cost $100k–$300k (equipment + workover), while offshore ESP workovers commonly total low millions in remote/subsea settings; these figures exclude the additional cost of deferred production during rig mobilization. Taken together, the FM's earlier and more confident detection materially reduces both deferred-production losses and the probability of costly unplanned interventions, supporting a strong business case for deployment.

In summary, the ESP case study highlights the value of FMs in a high-stakes industrial context. The model provided earlier and more confident detection of a developing failure than traditional methods and did so by leveraging patterns learned from other equipment and simulations. This early warning could translate to proactive maintenance actions that save time and cost. It also demonstrates that even though the model is trained to be general, after fine-tuning, it can serve as an expert system on a specific asset, with the advantage of having broader "experience" built in.

For completeness, we note that this is one case study; results may vary in other cases. Some anomalies may be more subtle or faster-developing, challenging any model. However, this example provides a template for how the TSFM can be deployed and the type of benefits it can offer in APM workflows.

## Conclusion

We presented a comprehensive study on architecting an asset-specific TSFM and demonstrated its applications for APM tasks such as forecasting and anomaly detection. The proposed approach leverages a discrete-token transformer backbone that is pretrained on diverse rotating equipment data and then fine-tuned by asset type. Our experimental results indicate that this approach offers significant benefits: improved predictive accuracy and earlier anomaly detection compared to conventional models, as well as the convenience of a unified framework that can be repurposed across different equipment and tasks.

The TSFM was able to capture generic operational characteristics of rotating machinery through pretraining, and with minimal fine-tuning it adapted to the nuances of a specific ESP pump, yielding a 10% reduction in RMSE and 8% reduction in MAPE for forecasting, and an F1-score improvement from 0.72 to 0.80 in anomaly detection (relative to using the FM without fine-tuning). The architecture's use of discrete tokenization and multi-head attention proved effective in modeling complex temporal patterns and cross-sensor relationships. The field case study illustrated a tangible outcome: early warning of a pump failure, highlighting the model's potential to enable proactive maintenance and reduce downtime.

Our work is among the first to successfully bring together ideas from FMs and apply them to industrial time series data in a holistic way. We showed that even a moderately sized transformer can function as

a powerful general-purpose model for equipment monitoring. Moreover, by attaching task-specific heads, we turned the foundation into a multi-tool for APM – capable of forecasting future behavior and detecting anomalies in real-time.

TSFMs offer a promising avenue to unify and enhance APM. With the right architecture and training strategy, a single model can indeed generalize across multiple assets while capturing individual asset needs. The success of this initial implementation sets the stage for more sophisticated and larger-scale efforts, ultimately moving us closer to AI systems that can understand and manage the complex behaviors of industrial machinery with minimal human supervision. We believe such systems will be an integral part of the next generation of smart maintenance and operations in the energy industry and beyond.

## Acknowledgements

We would like to thank Shashi Menon, Oleg Medvedev, and Arvind Sharma for their guidance and support on the project.

## References

1. Liang, Y., Wen, H., Nie, Y., Jiang, Y., Jin, M., Song, D., … & Wen, Q. (2024, August). Foundation models for time series analysis: A tutorial and survey. In Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining (pp. 6555–6565).
2. Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., & Sun, L. (2022). Transformers in time series: A survey. arXiv preprint arXiv:2202.07125.
3. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, **30**.
4. Biloš, M., Rasul, K., Schneider, A., Nevmyvaka, Y., & Günnemann, S. (2023, July). Modeling temporal data as continuous functions with stochastic process diffusion. In *International Conference on Machine Learning* (pp. 2452–2470). PMLR.
5. Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., … & Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.
6. Brooks, T., Peebles, B., Holmes, C., DePue, W., Guo, Y., Jing, L., Schnurr, D., Taylor, J., Luhman, T., Luhman, E., Ng, C., Wang, R., & Ramesh, A. (2024). *Video generation models as world simulators.* OpenAI. https://openai.com/research/video-generation-models-as-world-simulators.
7. Shi, X., Wang, S., Nie, Y., Li, D., Ye, Z., Wen, Q., & Jin, M. (2024). Time-moe: Billion-scale time series foundation models with mixture of experts. *arXiv preprint arXiv:2409.16040*.
8. Zerveas, G., Jayaraman, S., Patel, D., Bhamidipaty, A., & Eickhoff, C. (2021, August). A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 2114–2124).
9. Ansari, A. F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., … & Wang, Y. (2024). Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*.
10. Das, A., Kong, W., Sen, R., & Zhou, Y. (2024, July). A decoder-only foundation model for time series forecasting. In *Forty-first International Conference on Machine Learning*.
11. Ekambaram, V., Jati, A., Dayama, P., Mukherjee, S., Nguyen, N., Gifford, W. M., … & Kalagnanam, J. (2024). Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. *Advances in Neural Information Processing Systems*, **37**, 74147–74181.
12. Rasul, K., Ashok, A., Williams, A. R., Ghonia, H., Bhagwatkar, R., Khorasani, A., … & Rish, I. (2023). Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*.

13. Yu, Y., Zhu, Y., Wan, D., Zhao, Q., & Liu, H. (2019). A novel trend symbolic aggregate approximation for time series. *arXiv preprint arXiv:1905.00421*.
14. Xu, J., Wu, H., Wang, J., & Long, M. (2021). Anomaly transformer: Time series anomaly detection with association discrepancy. *arXiv preprint arXiv:2110.02642*.
15. Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International journal of forecasting*, **37**(4), 1748–1764.
16. Gao, S., Koker, T., Queen, O., Hartvigsen, T., Tsiligkaridis, T., & Zitnik, M. (2024). Units: A unified multi-task time series model. *Advances in Neural Information Processing Systems*, **37**, 140589–140631.
17. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
18. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, **32**.